

END SEM BLUEPRINT

Data Structures & Algorithms – II · BCSE0401 / BCS301

B.Tech 4th Sem · NIET Greater Noida · All 5 Units · Final Version

What your 5 units actually cover — verified from every slide:

Unit 1: Trees — BST, AVL (LL/LR/RL/RR rotations), B-Trees, Binary Heaps, Heap Sort

Unit 2: Graphs — Terminology, Adjacency Matrix/List, DFS, BFS, Spanning Trees, Prim's, Kruskal's, Dijkstra, Bellman-Ford, Floyd-Warshall

Unit 3: Dynamic Programming — 0/1 Knapsack, LCS, Floyd-Warshall, Matrix Chain Multiplication, Assembly Line Scheduling

Unit 4: Backtracking & Branch and Bound — N-Queens, Graph Coloring, Subset Sum, Hamiltonian Cycle, TSP

Unit 5: Advanced Data Structures — Red-Black Trees, B+ Trees, Binomial Heaps, Fibonacci Heaps, Tries

■ At-a-Glance: All 5 Units Ranked

1	Trees: BST, AVL, B-Tree, Heap Sort	Medium	■■■■■	Most repeatable — every step in AVL + BST is predictable with practice
2	Graphs: DFS/BFS, Prim, Kruskal, Dijkstra, Bellman-Ford	Medium	■■■■■	High-scoring — algorithm traces with diagrams are very doable if practiced once
3	Dynamic Programming: Knapsack, LCS, MCM, Floyd	Hard	■■■■	Cannot memorise — must practice filling tables. High reward if you do.

4	Backtracking & B&B: N-Queens, TSP, Graph Coloring	Medium-Hard	■■■■■	N-Queens solution is identical every time. TSP is harder but expected in slides.
5	Advanced DS: Red-Black, B+, Binomial/Fibonacci Heaps, Tries	Hard	■■■	No explicit expected questions in slides. Focus on properties only. Max 1.5 days.

■ UNIT 1 — Trees

The Unit 1 slides explicitly list "Expected Questions for University Exam" and weekly assignments with specific numbers to insert. Traversal construction, AVL rotations, and B-Tree insertions are the most reliably scored 10-mark questions in this subject.

■ High Priority — Do these numerically

- Tree traversals: Inorder (LNR), Preorder (NLR), Postorder (LRN) — trace on tree
- Construct binary tree from Inorder + Preorder or Inorder + Postorder
- BST: insert 43,10,79,90,12,54,11,9,50 step by step with diagram
- BST deletion — all 3 cases: leaf node, one child, two children (replace with inorder successor/predecessor)
- AVL Tree: insertions with LL, LR, RL, RR rotations clearly shown
- Build AVL from: a, z, b, y, c, x, d, w, e, v, f (from Expected Questions)
- B-Tree of order 4: insert a g f b k d h m j e s i r x c l n t u p
- B-Tree of order 5: insert 3,14,7,1,8,5,11,17,13,6,23,12,20,26,4,16,18,24,25,19; delete 6,23,3
- Heap Sort: algorithm + trace on A=[7,4,3,1,2]. State time complexity.

■ Quick Wins — Definitions only

- Tree terminology: Root, Parent, Child, Sibling, Leaf, Degree, Height, Depth, Path
- Full Binary Tree theorem: $L=I+1$, $N=2I+1$, $N=2L-1$
- Full vs Strictly Binary vs Complete vs Extended Binary Tree — one line each
- Array representation: parent at i , left child at $2i+1$, right child at $2i+2$
- Linked List representation — node structure (left, data, right)
- AVL Balance Factor = $\text{height}(\text{left subtree}) - \text{height}(\text{right subtree})$
- Priority Queue — what it is and how Binary Heap implements it

■ Expected Questions — Unit 1 (directly from slides)

--	--	--

1	Define extended binary tree, full binary tree, strictly binary tree and complete binary tree with examples.	5 M
2	Explain Threaded Binary Tree. Can a unique tree be determined from two traversals? Explain when yes and when no.	5 M
3	Construct binary tree from: INORDER: H K D B I L E A F C M J G PREORDER: A B D H K E I L C F G J M. Also find the Postorder traversal.	10 M
4	What is a B-Tree? Generate a B-Tree of order 4 for: a g f b k d h m j e s i r x c l n t u p	10 M
5	Draw expression tree for: $A * B - (C + D) * (P / Q)$. Write all three traversals.	7 M
6	What is a height-balanced (AVL) tree? Why is height balancing needed? Create AVL Tree for: a, z, b, y, c, x, d, w, e, v, f. Show all rotations.	10 M
7	Construct B-tree of order 5 inserting: 3,14,7,1,8,5,11,17,13,6,23,12,20,26,4,16,18,24,25,19. Then delete elements 6, 23 and 3.	10 M
8	Create BST using 28,56,12,9,87,7,23,19. Then delete node 56. Show all steps with diagrams.	10 M
9	Create AVL Tree using 39,56,2,9,87,7,23,19. Show all LL/LR/RL/RR rotations with balance factors.	10 M
10	Explain Heap Sort algorithm (Build-Heap + Heapify). Trace on array [7,4,3,1,2]. What is the time complexity?	10 M
11	Prove: In a non-empty full binary tree, number of leaves = number of internal nodes + 1.	5 M
12	Post Order traversal of a BST is: 10,9,23,22,27,25,15,95,60,40,29. Find the Preorder traversal.	7 M

■ UNIT 2 — Graphs & Graph Algorithms

Updated Unit 2 — this is now Graphs, NOT DBMS.

The new Unit 2 file covers: Graph types and terminology, Adjacency Matrix & Adjacency List representations, DFS, BFS, Connected Components, Spanning Trees, Prim's algorithm, Kruskal's algorithm, Dijkstra's algorithm, Bellman-Ford algorithm, and Floyd-Warshall algorithm. Every single algorithm here can be asked as a 10-mark trace numerical — that is your focus.

■ High Priority — Trace algorithms step-by-step

- DFS: algorithm + step-by-step trace ($S \rightarrow A \rightarrow D \rightarrow B \rightarrow C$ example from slides)
- BFS: algorithm + step-by-step trace with queue ($S \rightarrow A \rightarrow B \rightarrow C \rightarrow D$ example)
- DFS vs BFS comparison table — always asked
- Kruskal's: sort edges by weight, add if no cycle — trace on a weighted graph
- Prim's: grow tree from source, pick min-weight fringe edge — trace step by step
- Dijkstra's: single-source shortest path — trace with distance table and visited set
- Bellman-Ford: relax all edges $V-1$ times — trace iteration by iteration
- Floyd-Warshall: all-pairs shortest path $D[][]$ matrix update for each intermediate vertex
- Adjacency Matrix vs Adjacency List — space complexity + which is better when

■ Quick Wins — Learn these in 30 minutes

- Graph definition: $G = (V, E)$. Vertex, Edge, Path, Cycle, Degree definitions
- In-degree, Out-degree for directed graphs
- Types of graphs: Null, Trivial, Undirected, Directed, Connected, Disconnected
- Spanning Tree: N vertices $\rightarrow N-1$ edges, no cycle, connected
- MST properties: minimum total weight spanning tree
- Cayley's formula: complete graph with N vertices has N^{N-2} spanning trees
- Dijkstra cannot handle negative weight edges (Bellman-Ford can)
- Transitive Closure — reachability matrix concept
- DAG (Directed Acyclic Graph) — definition and use cases

■ Expected Questions — Unit 2 (from slides + standard exam pattern)

1	What is a Graph? Explain different types of graphs with examples (Directed, Undirected, Connected, Disconnected, Null, Trivial).	7 M
2	Explain Adjacency Matrix and Adjacency List representations of a graph. Compare their space and time complexity.	7 M

3	Explain DFS algorithm with example. Trace DFS starting from vertex S on the given graph. Use a stack.	10 M
4	Explain BFS algorithm with example. Trace BFS starting from vertex S on the given graph. Use a queue.	10 M
5	What is the difference between DFS and BFS? Compare them on: data structure used, time complexity, applications, and output.	7 M
6	What is a Minimum Spanning Tree? Explain Kruskal's algorithm step-by-step. Find MST of the given weighted graph.	10 M
7	Explain Prim's algorithm for finding MST. Trace on the given weighted graph. How does it differ from Kruskal's?	10 M
8	Explain Dijkstra's Algorithm for Single Source Shortest Path. Trace on a given weighted directed graph showing distance table at each step.	10 M
9	Explain Bellman-Ford Algorithm. When should it be used instead of Dijkstra's? Trace on a graph with negative weight edge.	10 M
10	Explain Floyd-Warshall Algorithm for All-Pairs Shortest Path. Trace through each intermediate vertex on the given graph.	10 M
11	What is a Connected Component? Write the algorithm to find number of connected components in a graph using DFS.	5 M
12	What is a DAG (Directed Acyclic Graph)? Construct a DAG for: $(a+b) \times (a+b+c)$. Explain construction rules.	5 M

■ UNIT 3 — Dynamic Programming

Honest warning about Unit 3:

This is the hardest unit. Every problem requires building and filling a DP table by hand. You cannot memorise your way through this — table-filling must be practiced. The good news: the same 4-5 problems appear every year and the steps are always identical. Practise each one at least 3 times on paper.

■ High Priority — Practice tables

- 0/1 Knapsack: $c[i][w]$ table. Given weights=[2,3,4,5], profits=[3,4,5,6], $W=5$ (from slides)
- Floyd-Warshall: $D[i][j]$ matrix update for each intermediate vertex k (also in Unit 2!)
- LCS of two strings: $S1=BCDAAC$, $S2=ACDBA$ — build table and backtrack
- Matrix Chain Multiplication: $m[i][j]$ table for $A(10 \times 30)$, $B(30 \times 5)$, $C(5 \times 60)$, $D(60 \times 8)$
- Assembly Line Scheduling: recurrence for $f1(j)$ and $f2(j)$
- DP properties: Optimal Substructure + Overlapping Subproblems — must explain these

■ Quick Wins — Understand concepts

- What is Dynamic Programming? When to use it?
- DP vs Greedy vs Divide and Conquer — 1-table comparison (commonly asked)
- Top-down (Memoization) vs Bottom-up (Tabulation)
- Fractional Knapsack is Greedy, NOT DP — trick MCQ
- LCS vs Substring: subsequence need not be contiguous
- MCM complexity: $O(n^3)$
- Knapsack complexity: $O(n \cdot W)$

■ Expected Questions — Unit 3

1	Explain 0/1 Knapsack using Dynamic Programming. Items: weights=[2,3,4,5], profits=[3,4,5,6], capacity $W=5$. Build the complete table and trace back the solution.	10 M
2	Find the LCS of $S1=\{B,C,D,A,A,C,D\}$ and $S2=\{A,C,D,B,A,C\}$ using DP. Show the complete LCS table and backtrack to find the actual subsequence.	10 M
3	Explain Matrix Chain Multiplication using DP. Find optimal parenthesization for $A(10 \times 30)$, $B(30 \times 5)$, $C(5 \times 60)$, $D(60 \times 8)$. Show the $m[i][j]$ table.	10 M
4	Solve Floyd-Warshall All-Pairs Shortest Path on the given weighted graph. Show $D[0]$, $D[1]$, ... $D[k]$ at each step.	10 M
5	What are the two key properties a problem must have for DP to apply? Explain Optimal Substructure and Overlapping Subproblems with examples.	7 M
6	Write the difference between Greedy method and Dynamic Programming. Give one problem where Greedy fails but DP works.	7 M

7	Explain Assembly Line Scheduling using DP. Write the recurrence relations for $f1(j)$ and $f2(j)$. Solve a given numerical.	7 M
8	Write the algorithm for 0/1 Knapsack DP. What is its time and space complexity? Why can't Fractional Knapsack be solved by DP?	5 M

UNIT 4 — Backtracking & Branch and Bound

Unit 4 has an explicit "Expected Questions" section in the slides with 6 specific questions. All 6 are listed below. N-Queens and TSP are the most exam-frequent. Drawing the state-space tree clearly and labelling pruned nodes is what earns marks here.

High Priority — Draw state-space trees

- N-Queens: 4-Queen problem — draw complete state-space tree and show solution (2,4,1,3) and (3,1,4,2)
- 8-Queen: at least show one valid solution path through backtracking
- TSP using Branch & Bound: cost matrix reduction, row/column minimum subtraction, node cost calculation
- Graph Coloring: assign m colors to vertices such that no two adjacent vertices share color — trace with backtracking
- Subset Sum: given $S=(3,4,5,6)$ $X=9$ — draw implicit binary tree, identify solutions
- Hamiltonian Cycle: definition + example of when it exists vs when it doesn't
- State-Space Tree: what it is + how it is traversed (always DFS in backtracking)

Quick Wins — Conceptual answers

- Backtracking definition: build solution incrementally, abandon when constraint fails
- Three problem types: Decision, Optimisation, Enumeration
- Pruning: bounding function + when to cut a subtree
- Branch & Bound vs Backtracking: B&B uses cost bound, backtracking uses feasibility
- Why is TSP NP-hard? (Exponential combinations)
- Greedy vs DP vs Backtracking — one-line comparison
- Hamiltonian vs Eulerian path distinction

Expected Questions — Unit 4 (directly from slides)

1	Solve the Travelling Salesman Problem using Branch and Bound on the given cost matrix. Show all node expansions and cost calculations.	10 M
2	Write short notes on: (i) Graph Coloring Problem (ii) Subset Sum Problem (iii) Hamiltonian Cycle	10 M
3	Find one optimal solution using Backtracking for: (a) 4-Queen Problem on 4x4 board (b) 8-Queen Problem on 8x8 board. Draw the state-space tree.	10 M
4	Write the difference between Greedy method and Dynamic Programming with examples.	5 M
5	Explain the term State-Space Search Tree with a suitable example. How is it traversed in backtracking?	7 M
6	Explain Floyd-Warshall Algorithm and analyze its time complexity. [Note: This is also in Unit 2 and Unit 3]	7 M

7	What is Backtracking? Explain with algorithm and example. When does backtracking occur?	7 M
8	Given $S=\{3,4,5,6\}$ and $X=9$, solve Subset Sum Problem using Backtracking. Draw the complete implicit binary tree.	10 M
9	Explain Graph Coloring Problem using Backtracking. Write algorithm and apply for a 4-vertex graph with $m=3$ colors.	7 M

■ UNIT 5 — Advanced Data Structures

Honest assessment of Unit 5:

Unit 5 slides (176 slides!) have NO explicit "Expected Questions" section. This means expected questions are inferred from MCQs and the slide content. Focus on: Red-Black Tree 5 properties + height theorem (these appear in MCQs), B+ Tree vs B-Tree difference (asked in almost every advanced DS exam), and Tries (definition + search/insert). Do NOT try to memorise full Fibonacci Heap algorithms — it is not worth the time. **Spend max 1.5 days here and move on.**

■ High Priority — Learn these properly

- Red-Black Tree: all 5 properties — memorise word-for-word (Every node red/black; root black; leaves NIL black; red node has black children; all paths same black-height)
- RB Tree height theorem: $h \leq 2\log_2(n+1)$ — learn the proof steps
- Left-Rotate and Right-Rotate operations — algorithm + diagram
- Black-height $bh(x)$: number of black nodes from x to leaf, NOT counting x
- B-Tree vs B+ Tree: key difference — B+ stores all data at leaf level, leaves are linked
- Tries: definition, insert, search operations with example
- Binomial Heap: what is a binomial tree B_k ? List of properties

■ Quick Wins — One-liners

- Why RB Tree instead of BST? $O(\log n)$ guaranteed in worst case vs $O(n)$
- Why RB Tree vs AVL? RB Tree rebalances less aggressively — faster insertions
- B-Tree of order m : each non-root has $\lceil m/2 \rceil - 1$ to $m-1$ keys
- B+ Tree: internal nodes hold only keys for routing; all actual data in leaf nodes
- Fibonacci Heap advantage: decrease-key $O(1)$ amortized (vs $O(\log n)$ in binary heap)
- Trie applications: autocomplete, spell check, IP routing, dictionary lookups
- Sentinel $\text{nil}[T]$ in RB Tree: single black node representing all leaves

■ Most Likely Questions — Unit 5

1	What is a Red-Black Tree? List all 5 properties with explanation. Prove that height of RB Tree with n nodes is at most $2\log_2(n+1)$.	10 M
2	Explain Left-Rotate and Right-Rotate operations in a Red-Black Tree. Write the algorithm and show with a diagram.	7 M
3	What is a B+ Tree? How is it different from a B-Tree? Draw an example. What are its advantages in database indexing?	7 M

4	What is a Trie data structure? Explain insert and search operations with an example. What are its real-world applications?	7 M
5	Explain Binomial Heaps. What is a binomial tree B_k ? List properties. How is a Binomial Heap implemented?	7 M
6	Compare Binary Heap, Binomial Heap, and Fibonacci Heap — time complexities for: insert, extract-min, decrease-key, union.	7 M
7	What is black-height of a Red-Black Tree? Show an example RB Tree and compute black-height of each node.	5 M
8	Why are Red-Black Trees preferred over AVL Trees for general use? What is the trade-off?	5 M

■ 10-Day Preparation Schedule

Time allocation logic — be honest with yourself:

Units 1 and 2 together can give you the most marks if you do algorithm traces well. Unit 3 (DP) takes the longest but pays off. Unit 4 is learnable fast — the same 4 problems repeat. Unit 5 is the lowest return on time invested — cap it at 1.5 days.

1	Unit 1 — Traversals + BST	Learn all 3 traversal algorithms. Practice 6 traversal traces. Build binary tree from Inorder+Preorder (HKDBILEAFCMJG example). BST: insert 43,10,79,90,12,54,11,9,50.	5 hrs
2	Unit 1 — AVL + B-Tree + Heap	All 4 AVL rotations (LL/LR/RL/RR). Build AVL for a,z,b,y,c,x,d,w,e,v,f. B-Tree order 4 insertion. Heap Sort algorithm + trace on [7,4,3,1,2].	5 hrs
3	Unit 2 — Graph basics + DFS + BFS	Graph types + terminology. Adjacency Matrix vs Adjacency List (code + properties). DFS trace (S→A→D→B→C). BFS trace with queue. DFS vs BFS comparison table.	5 hrs
4	Unit 2 — MST + Shortest Paths	Kruskal's: sort edges, add without cycle. Prim's: grow from source. Practice both on a 6-node weighted graph. Dijkstra: distance table trace. Bellman-Ford: V-1 iterations.	5 hrs
5	Unit 2 + Unit 3 — Floyd-Warshall + DP intro	Floyd-Warshall: D[i][j] update for each k (this appears in Units 2, 3 AND 4 — learn it ONCE properly). Then: What is DP? DP vs Greedy. Knapsack problem — fill first table.	5 hrs
6	Unit 3 — DP: LCS + MCM	LCS table for S1=BCDAACD, S2=ACDBAC. Backtrack to find actual subsequence. Matrix Chain Multiplication: fill m[i][j] for 4 matrices. Assembly Line Scheduling recurrence.	5 hrs
7	Unit 4 — Backtracking	4-Queen and 8-Queen: draw complete state-space trees. Subset Sum S={3,4,5,6} X=9: binary tree. Graph Coloring: 4-vertex example with 3 colors. Hamiltonian Cycle: definition + small example.	4 hrs
8	Unit 4 — TSP + Unit 5 start	TSP using Branch and Bound: cost matrix reduction, row subtraction, node cost formula. Then start Unit 5: RB Tree — 5 properties + height theorem. Memorise them.	4 hrs
9	Unit 5 — B+ Tree + Tries + Heaps	B+ Tree vs B-Tree: draw comparison table + example. Trie: insert and search for 5 words. Binomial Heap: what is Bk tree, list properties. Fibonacci Heap: just amortized complexities.	4 hrs
10	Full revision + mock paper	Attempt 2 previous year papers under exam conditions. Mark weak spots. Redo those specific numericals. Review the Quick Reference Card below.	5 hrs

Days 3 and 6 (highlighted) are the most critical — do not compress them.

■ Quick Reference — Formulae & Algorithms to Memorise

Max nodes at level l (BT)	2^l (root is level 0)
Max nodes in BT, height h	$2^{(h+1)} - 1$
Min height for N nodes	$\text{floor}(\log_2(N+1)) - 1$
Full BT theorem	$L = l+1 \mid N = 2l+1 \mid N = 2L-1$
Heap Sort complexity	$O(n \log n)$ — worst, average, best (all same)
AVL Balance Factor	$BF = \text{height}(\text{left}) - \text{height}(\text{right}) \mid \text{Valid range: } -1, 0, +1$
B-Tree order m (non-root)	Min keys: $\text{ceil}(m/2)-1$ Max keys: $m-1$
Spanning tree edges	Always $N-1$ edges for N vertices
Kruskal complexity	$O(E \log E)$ — dominates due to edge sorting
Prim complexity	$O(V^2)$ with adjacency matrix $O(E \log V)$ with min-heap
Dijkstra complexity	$O(V^2)$ — cannot handle negative weight edges
Bellman-Ford complexity	$O(VE)$ — handles negative edges, detects negative cycles
Floyd-Warshall complexity	$O(V^3)$ — all-pairs, update: $D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$
Knapsack recurrence	$c[i, w] = \max(c[i-1, w], v_i + c[i-1, w-w_i])$ if $w_i \leq w$
LCS recurrence	$L[i, j] = L[i-1, j-1] + 1$ if $X_i = Y_j$ else $\max(L[i-1, j], L[i, j-1])$
MCM complexity	$O(n^3)$ — three nested loops
N-Queens (4x4) solutions	(2,4,1,3) and (3,1,4,2) — memorise both
RB Tree: 5 properties	(1) Red/Black (2) Root=Black (3) Leaves NIL=Black (4) Red node has Black children (5) Equal black-height on all paths
RB Tree height bound	$h \leq 2 * \log_2(n+1)$
Black-height bh(x)	Black nodes from x to leaf, NOT counting x
B+ Tree key difference	All data at leaf level; leaves are doubly-linked list
Cayley's formula (ST)	$N^{(N-2)}$ spanning trees in complete graph with N vertices

■ Scoring Strategy — Read This Before the Exam

■ Biggest mistake to avoid:

Spending too long on Unit 5 (Advanced DS) or trying to memorise Fibonacci Heap insertion algorithms. The slides for Unit 5 have zero explicit expected questions. Instead, use that time to do one extra practice of Kruskal's or Dijkstra's — those algorithms will appear in Unit 2 questions and score you 10 marks.

■ Where you can guarantee marks fastest:

Unit 2 is your biggest opportunity in this revised blueprint. DFS, BFS, Kruskal's, and Dijkstra's traces are mechanical — same steps every time. Practice each once on paper and you will nail the 10-mark questions. Combined with Unit 1 tree numericals, these two units alone can give you 40+ marks.

■ Floyd-Warshall appears in THREE units:

This is important — Floyd-Warshall is covered in Unit 2 (Shortest Paths), Unit 3 (Dynamic Programming), AND Unit 4 (Expected Questions). This means it is VERY likely to be on your exam. Learn it once, learn it properly. Know the $D[i][j]$ matrix update formula: $D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$.

■ How to structure 10-mark answers in this subject:

(1) Definition — 2 lines. (2) Algorithm — clearly numbered steps. (3) Trace on example — clean diagram with each step labelled. (4) Time Complexity — state it. This structure earns 8-9/10 even with minor errors. Examiners mark on structure first.

■ The single most important thing:

Draw clean, labelled diagrams. In this subject — AVL rotations, BFS/DFS step traces, Kruskal edge additions, Dijkstra distance tables, state-space trees, DP tables — everything is visual. A neat diagram with clear labels will earn you marks even if your written explanation is incomplete. Bring a ruler to the exam and draw proper boxes for DP tables.

■ All 5 units done. You have everything. Now it is just about practice — do the numericals.